

ZAATOT

Backend Architecture and Complete Project Guide

A file-by-file repository audit, technology rationale, system walkthrough, risk review, and tailored learning roadmap.

153 tracked files | 21,079 logical lines including generated metadata | 81 source files | 10 database tables | 39 HTTP surfaces including development routes | 71 test cases

Prepared from the working repository snapshot

28 August 2026 - Africa/Casablanca

Contents

This guide is designed to work in three modes: a fast executive read, a technical reference, and a structured curriculum for learning the codebase.

Contents	2
1. Executive orientation	4
1.1 Current maturity assessment	4
1.2 The five ideas to learn first	4
1.3 Audit method and limits	4
2. System architecture	5
2.1 Architectural style and philosophy	5
2.2 Control plane and tenant data planes	6
2.3 Request lifecycle	6
2.4 Startup and shutdown	6
2.5 Repository structure	7
3. Technology stack: purpose, philosophy, and alternatives	8
3.1 Bun 1.3.14: runtime, package manager, test runner, bundler, and compiler	8
3.2 TypeScript 6.0.3: executable design constraints	8
3.3 Elysia 1.4.28: typed HTTP composition	8
3.4 Elysia JWT, CORS, Swagger, and Eden	9
3.5 Turso/libSQL: globally reachable SQLite with tenant isolation	9
3.6 Drizzle ORM 0.45.2 and Drizzle Kit 0.31.10	9
3.7 Redis through Bun RedisClient	9
3.8 Cloudflare R2 and AWS SDK v3	10
3.9 AI SDK, Google provider, VoltAgent, and Zod	10
3.10 Docker, Compose, GitHub Actions, Prettier, and custom rules	10
3.11 Direct dependency resolution and actual use	11
4. Database and tenancy deep dive	12
4.1 Main database tables	12
4.2 Tenant tables	12
4.3 Connection routing and LRU cache	12
4.4 Provisioning lifecycle	13
4.5 Main migrations	13
4.6 Custom tenant schema synchronization	13
4.7 Generic select engine	13
5. Domain modules and business flows	14
5.1 Authentication flow	14
5.2 Organization and tenant ownership	14
5.3 Contacts	15
5.4 Files and object storage	15

5.5 Product catalog and Open Food Facts	15
5.6 Scan orchestration	15
5.7 Two boycott decision paths	16
5.8 AI foundation	16
6. HTTP API inventory	17
6.1 Query contract	17
7. Configuration, build, and delivery	19
7.1 Environment variables	19
7.2 Package scripts	19
7.3 TypeScript and formatting philosophy	19
7.4 CI and deployment	20
8. Code-quality system and tests	21
8.1 Executable rules	21
8.2 Test coverage shape	21
9. Security, reliability, and production-readiness review	22
9.1 Security mechanisms already present	22
9.2 Observability design to add	22
9.3 Production rollout order	23
10. Tailored learning roadmap	24
10.1 Concepts to go deep on	24
10.2 Recommended reading order inside this repository	24
10.3 Practical exercises	25
11. Project-management view	26
11.1 Suggested workstreams	26
11.2 A realistic delivery sequence	26
11.3 Definition of done for backend changes	26
12. File-by-file repository appendix	27
13. Source notes and official references	31
13.1 Glossary	31

1. Executive orientation

One-sentence description

Zaatot is a Bun and Elysia API that scans or identifies products, enriches them from external providers, evaluates boycott information, records user activity, and isolates private user and organization data in separately provisioned Turso databases.

The project is best understood as a control plane plus many data planes. The main Turso database is the control plane: it knows users, organizations, memberships, products, and brands. Each user and organization receives a separate Turso database that stores tenant-private data. Redis coordinates short-lived and distributed state. Cloudflare R2 stores file bytes. Elysia exposes the HTTP contract, and Drizzle translates typed TypeScript schemas and query expressions into SQLite/libSQL operations.

The architecture is intentionally modular. A controller speaks HTTP, a DTO defines the runtime and compile-time contract, and a service owns business logic and database interaction. Shared libraries provide errors, authentication, middleware, enums, environment validation, and generic query construction. Custom TypeScript AST rules make many of these conventions executable rather than relying only on documentation.

The product domain is already more than CRUD. A barcode scan first checks the global product cache, then calls Open Food Facts, normalizes brand and nutrition data, stores the result, writes a per-user scan-history record, and optionally asks Boycat for boycott evidence. A second, internal boycott-decision engine uses a curated seed, entity normalization, alias and relationship matching, source trust, claim confidence, and campaign tiers.

1.1 Current maturity assessment

Area	Assessment	Why
Architecture intent	Strong	Clear boundaries, database isolation, typed contracts, executable conventions, and explicit external adapters.
Feature maturity	MVP / early beta	Core flows exist, but OTP delivery is a console placeholder, AI agents are not wired, and operational reconciliation is incomplete.
Type safety	Strong at boundaries	Strict TypeScript and Elysia schemas are used broadly; several any casts and database enum gaps weaken the final guarantee.
Data safety	Needs hardening	Custom tenant schema sync can perform destructive changes, swallows some errors, and is not transactional.
Security	Good foundation, important gaps	JWT, rate limits, headers, tenant ACLs, and file restrictions exist; global mutations lack role checks and upload claims are client-controlled.
Testing	Useful unit coverage	71 mock-heavy tests cover happy paths and selected failures, but there are no live database, boot, container, or external contract tests.
Operations	Basic	Multi-stage images, CI, graceful shutdown, and env validation exist; observability, health/readiness, backups, provisioning repair, and deployment automation are not implemented.

1.2 The five ideas to learn first

- Request lifecycle: understand Elysia plugin composition, lifecycle hooks, DTO validation, authentication derivation, and centralized errors.
- Database-per-tenant: understand why the main database and tenant databases have different responsibilities and failure modes.
- Service-oriented modules: trace controller -> DTO -> service -> schema/adapters for one simple module before reading the scan orchestration.
- Distributed coordination: see how Redis supports OTPs, rate limiting, access caching, and schema migration locks.
- Evidence-based decisions: separate raw provider responses, normalized evidence, matching, confidence calculation, and the final user-facing decision.

1.3 Audit method and limits

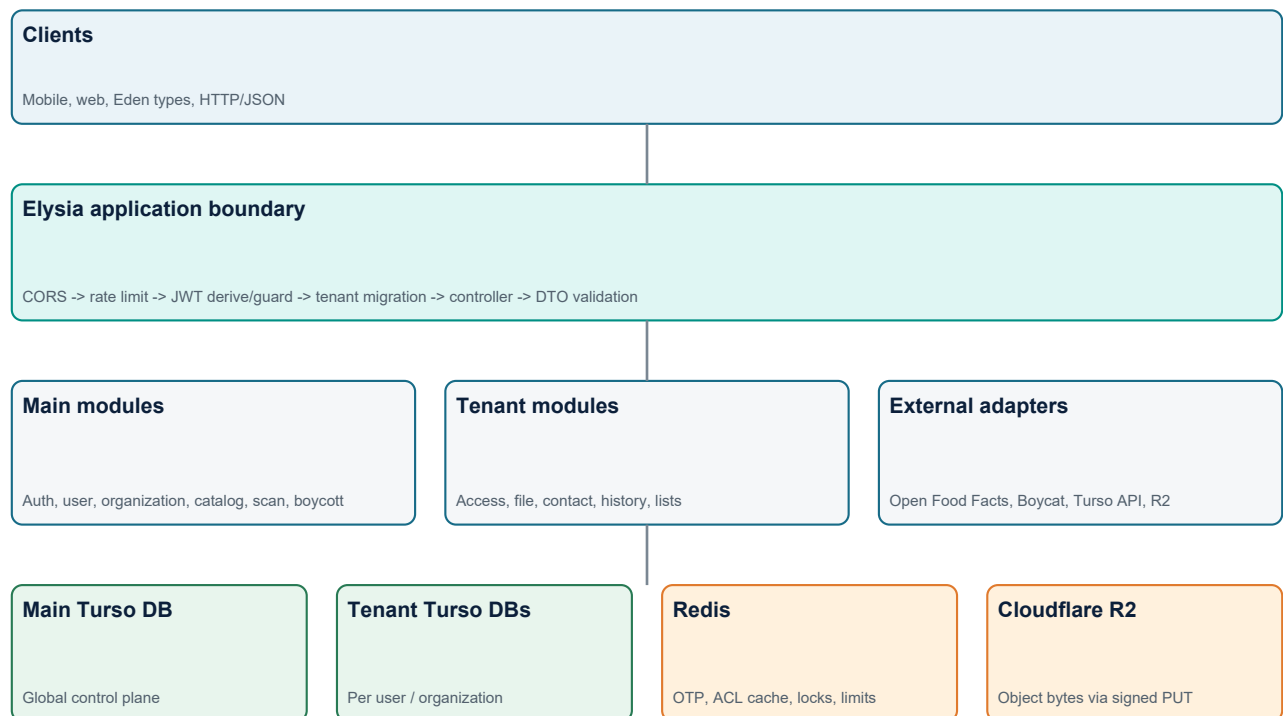
Every tracked file was opened during the audit. Handwritten source, configuration, project instructions, custom rules, SQL migrations, tests, and deployment files were reviewed semantically. Machine-generated Drizzle snapshots and the Bun lockfile were analyzed structurally: schema history, direct dependency resolution, workspace metadata, and transitive dependency shape were checked without reproducing thousands of generated lines in this guide.

This is a static repository audit, not a penetration test or a live production assessment. Bun, Docker, dependencies, credentials, and backing services were not available in the analysis environment, so the CI scripts and 71 tests could be inspected but not executed. Recommendations that depend on production traffic must be validated with metrics and load testing.

Snapshot note

The report reflects the working tree observed on 28 August 2026. Generated and actively edited files may change after this snapshot; use the repository as the final source of truth.

2. System architecture



2.1 Architectural style and philosophy

The project guidance names the style a modular service-oriented architecture. It is modular because features are organized by business capability rather than technical file type alone. It is service-oriented because business operations live in named service objects that can be called from HTTP controllers, AI tools, tests, or other services. It is not a microservice system: all modules are compiled into one process and deployed together.

This middle position is often called a modular monolith. It keeps local calls, one deployment, one type system, and straightforward debugging while preserving enough boundaries to extract a high-pressure module later. For the current team and product stage, this is usually cheaper and safer than premature microservices. Microservices would become attractive only when independent scaling, ownership, release cadence, or regulatory isolation outweigh distributed-system complexity.

Stateless request handlers are another core choice. Durable state lives in Turso, Redis, or R2 rather than process memory. That makes horizontal replication possible. The in-process tenant connection map is an optimization, not business state; losing it only forces connections to be recreated.

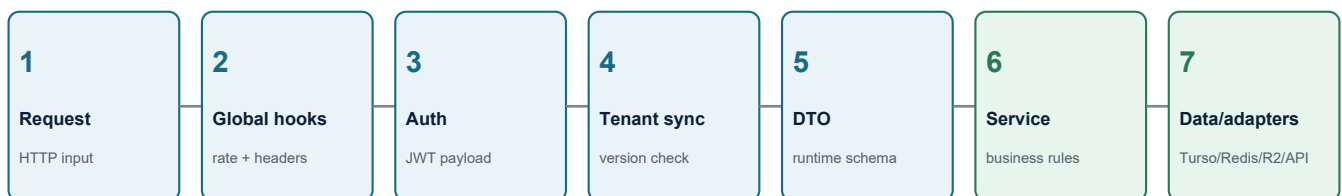
2.2 Control plane and tenant data planes

Boundary	Owns	Does not own	Why
Main database	Users, organizations, memberships, global products and brands, tenant DB identifiers and schema versions	Contacts, files, per-user history and lists	Global discovery and account coordination need one consistent catalog.
Common tenant schema	Access records and file metadata	Global identity or catalog data	Every tenant needs authorization and file capabilities.
Organization database	Common tenant tables plus contacts	Other organizations or user personalization	Stronger tenant isolation and simpler tenant-level export/deletion.
User database	Common tenant tables plus scan history and white/black lists	Global product truth	Personal history stays isolated while products remain deduplicated globally.
Redis	Short-lived OTPs, distributed counters, ACL cache, migration locks	Authoritative durable business records	Fast expiry and atomic primitives suit coordination state.
R2	Object bytes	Relational metadata or access policy	Large unstructured media belongs outside SQL.

Isolation tradeoff

Database-per-tenant gives strong blast-radius and export boundaries, but it moves complexity into provisioning, connection management, schema rollout, cross-tenant analytics, backups, and repair. The project accepts those operational costs deliberately.

2.3 Request lifecycle



Errors from any stage are normalized by `handle_error`; response schemas are validated by Elysia.

At startup, environment validation runs before the rest of the app. Elysia is configured with an idle timeout, CORS, development-only Swagger, JWT support, a global IP rate limiter, response security headers, and centralized error mapping. Public health-like HTML and Eden type routes are registered, then the public authentication controller is mounted.

All other controllers are placed inside a derived and guarded group. The group verifies the bearer token, exposes a typed payload to downstream handlers, and checks whether a tenant mentioned in params, query, or body needs a schema migration. A successful lazy migration updates the schema version embedded in the JWT and returns a replacement token in `X-Refresh-Token`.

Controllers stay thin: they extract body/query/payload and delegate. DTOs are attached directly to routes, so the same definition validates runtime input, constrains output, informs OpenAPI, and helps TypeScript infer handler types. Services then apply authorization, business rules, database calls, caching, or provider integration.

2.4 Startup and shutdown

- Startup fails fast when required configuration is missing. This avoids running a partially configured process.
- The server listens on PORT, defaulting to 3000, and logs the resolved host and port.
- SIGTERM and SIGINT trigger an idempotent shutdown routine. The HTTP listener stops, cached libSQL clients and four Redis clients close, the S3 client is destroyed, and the process exits.
- A 30-second forced-exit timer prevents an indefinitely hung shutdown. This is a sensible container behavior, although in-flight request tracking and structured shutdown logs would improve it.

2.5 Repository structure

Path	Responsibility	Design meaning
src/app.ts	Composition root and server lifecycle	The only source file allowed at src root.
src/lib	Cross-cutting application primitives	Errors, DTO fragments, enums, env, JWT, middleware.
src/db	Connections, schemas, generic query and tenant schema sync	All database mechanics are centralized.
src/module/main	Global account, catalog, scan, and boycott capabilities	Uses the main control-plane database or external providers.
src/module/tenant	Capabilities common to every tenant	Access and file metadata.
src/module/organization	Organization-only capabilities	Contacts today; extensible later.
src/module/user	User-only capabilities	Scan history and personal lists.
src/storage	Object-storage client	S3-compatible R2 adapter.
src/ai	AI message/tool foundations	Scaffolding is present; production agents are not yet implemented.
rule	Executable architecture and naming checks	Project-specific static analysis supplements TypeScript and Prettier.
.drizzle	Main-database migration history	Generated SQL, snapshots, and migration journal.
.agents	Contributor/agent design contract	Explains intended architecture and required implementation patterns.

3. Technology stack: purpose, philosophy, and alternatives

No technology is universally better. In this section, 'better' means a stronger fit for this project's constraints: TypeScript end-to-end contracts, fast iteration, database-per-tenant isolation, small-team operations, external API orchestration, and future AI tooling.

3.1 Bun 1.3.14: runtime, package manager, test runner, bundler, and compiler

Bun is the platform foundation. The project uses Bun to run TypeScript directly in development, install from `bun.lock`, execute Jest-like tests, expose a built-in `RedisClient`, watch source files, and compile the application into a standalone Bun-targeted binary. This collapses several Node-era tools into one runtime.

The philosophy is cohesion and low tool overhead. A single executable understands packages, TypeScript transpilation, tests, bundling, scripts, file APIs, cryptography, Redis, and HTTP serving. That reduces configuration drift and makes the Docker release stage simple: it only copies the compiled binary into a Bun slim image.

Compared with Node.js, Bun is a better fit when startup speed, direct TypeScript, one-tool workflows, and Bun-native frameworks matter. Node remains better when the organization depends on the widest runtime compatibility, mature APM support, conservative long-term support policies, or native modules that have not been validated on Bun. Compared with Deno, Bun offers stronger npm/drop-in familiarity for this dependency set; Deno can be better when permission-based runtime security and web-standard-first tooling dominate.

The lockfile pins exact direct and transitive resolutions. One issue to fix is metadata drift: `package.json` names the project `zaatot-backend`, while the observed lockfile workspace name is `elysia-backend`. Regenerate the lockfile with the intended Bun version and review the diff.

3.2 TypeScript 6.0.3: executable design constraints

TypeScript is not used only for autocomplete. It is the language in which API schemas, database schemas, JWT payloads, service arguments, AI messages, and custom AST rules are expressed. The compiler configuration enables `strict`, `noUncheckedIndexedAccess`, `unused checks`, `switch fallthrough checks`, and `bundler-style resolution`.

The philosophy is to move defects left: represent invalid states as types where practical, then validate untrusted data at runtime. TypeScript alone disappears at runtime, so Elysia schemas and Zod schemas are essential partners rather than optional duplication.

Compared with plain JavaScript, TypeScript is clearly better for a growing service because refactors cross controllers, DTOs, services, tables, and clients. Compared with Go or Rust, TypeScript keeps one language across backend, generated Eden clients, and AI tooling and offers faster product iteration. Go or Rust would be stronger for predictable resource use, CPU-heavy work, or extremely high concurrency, but would add a language boundary and a different ecosystem.

Path aliases make imports architectural: `@db`, `@lib`, `@module`, `@storage`, and AI aliases describe ownership. The custom rule suite forbids relative imports, so moving a file is less likely to cause long relative-path rewrites.

3.3 Elysia 1.4.28: typed HTTP composition

Elysia is the web framework and the composition model. Each controller is an Elysia plugin with a prefix. The main app composes controllers, lifecycle hooks, JWT, CORS, and Swagger. The framework's `t` schema builder validates requests and responses and feeds type inference and OpenAPI.

The philosophy is one contract across runtime and compile time. Express usually requires separate validator, OpenAPI, and type wiring. Elysia makes the route schema part of the route itself. That is especially valuable here because almost every find endpoint exposes a structured query language with selectable columns, filters, pagination, ordering, grouping, and response shapes.

Compared with Express, Elysia is a better fit for Bun performance and integrated type contracts; Express is better for maximum ecosystem familiarity and middleware compatibility. Compared with Fastify, both support schema-driven performance, but Elysia offers a more Bun-native and tightly inferred model. Compared with NestJS, Elysia is far lighter and less ceremonial; Nest is better for very large teams that value dependency injection, decorators, and enforced enterprise layering.

A subtle but important concept is plugin lifecycle scope. Authentication is applied through a parent group and `lib_jwt` is also used in controllers. When extending the app, verify that new hooks are scoped to the intended descendants; Elysia lifecycle hooks are isolated by plugin scope unless exported/configured correctly.

3.4 Elysia JWT, CORS, Swagger, and Eden

`@elysiajs/jwt` signs one-day bearer tokens containing the user ID and a tenant list with schema versions. This is efficient and stateless, but token revocation, key rotation, refresh policy, and membership staleness are not implemented. Server-side sessions would simplify revocation and keep authorization current, at the cost of a database/cache lookup for each session.

`@elysiajs/cors` enables credentialed cross-origin requests and exposes the refresh-token header. The production origin allowlist is not explicit in the current app. Define exact frontend origins, allowed methods/headers, and proxy behavior before public deployment; credentials with permissive origins are a common security mistake.

`@elysiajs/swagger` is mounted only when `ENV=dev`. This is a sensible reduction in production surface. Protect or disable it in shared development environments if request models or internal endpoints are sensitive.

Eden provides end-to-end client types. The eden script asks TypeScript to emit only declarations for `src/app.ts`, copies the resulting app type to `public/eden.ts`, and the development server exposes it at `/eden`. This lets frontend code consume the server contract without manually duplicating endpoint types. The generated artifact is intentionally ignored by Git.

3.5 Turso/libSQL: globally reachable SQLite with tenant isolation

The project uses `@libsql/client 0.17.3` for SQL traffic and `@tursodatabase/api 1.9.2` to create databases programmatically. Turso gives the team a SQLite-compatible data model without operating a traditional database server. Each tenant database name encodes app, environment, tenant type, and tenant ID.

The philosophy is physical data isolation with a familiar relational model. Separate databases can reduce accidental cross-tenant queries, make tenant export/restore conceptually simpler, and bound the impact of a corrupted tenant database. It also aligns with a geographically distributed or serverless application that does not want to manage PostgreSQL clusters.

Compared with one PostgreSQL database plus row-level security, Turso database-per-tenant is stronger at physical separation and tenant-specific lifecycle. PostgreSQL is stronger for cross-tenant analytics, rich constraints, high write concurrency, complex joins, mature operational tooling, and one migration path. A schema-per-tenant PostgreSQL design sits between these choices.

Current Turso documentation describes `@libsql/client` as legacy but fully supported and battle-tested, and still the production-ready choice for mature ORM integration. That means the project is not immediately wrong, but the client choice should be reviewed periodically. The old Turso Multi-DB Schemas feature is deprecated; this project does not rely on it and instead performs its own schema synchronization.

3.6 Drizzle ORM 0.45.2 and Drizzle Kit 0.31.10

Drizzle defines SQLite tables in TypeScript, constructs typed SQL-like queries, and generates main-database migrations. Its design stays close to SQL: a developer reading `select().from().where()` can usually predict the generated statement.

Compared with Prisma, Drizzle has a thinner abstraction, less generated-client machinery, and more direct SQL control. Prisma has a more batteries-included schema language, studio, relation client, and broader team familiarity. Compared with raw SQL, Drizzle catches table/column mistakes earlier and composes filters safely, while raw SQL remains best for database-specific tuning or complex operations that an abstraction makes awkward.

The project uses two migration philosophies. The main database follows generated, versioned SQL in `.drizzle`. Tenant databases use a custom runtime diff against Drizzle table definitions. Mixing these is understandable because there may be thousands of tenant databases, but the custom path must be treated like a deployment system, not a helper function.

3.7 Redis through Bun RedisClient

Four logical Redis connections separate authentication OTPs, tenant access caching, migration locks, and rate-limiting counters. This separation can map to different endpoints, databases, credentials, or service tiers. Redis is chosen for expiration and atomic commands rather than durable relational truth.

Compared with process memory, Redis works across multiple application replicas and survives individual process restarts. Compared with storing every counter or OTP in SQL, Redis provides simple TTLs and lower coordination latency. The cost is another network dependency, failure-mode design, and a need to decide whether fail-open or fail-closed is correct for each feature.

The global limiter initializes a key with zero and TTL, then increments it. More advanced production limiting could use a Lua script or native rate-limit primitive to make window initialization and increment one atomic operation, return retry metadata, and avoid fixed-window bursts. IP extraction must also be proxy-aware; direct socket addresses collapse all users behind a reverse proxy unless trusted forwarded headers are handled deliberately.

3.8 Cloudflare R2 and AWS SDK v3

R2 stores images, audio, and video; tenant databases store only metadata. The backend generates a signed S3-compatible PUT URL so the client uploads bytes directly to R2. This avoids routing large files through the application and reduces CPU, memory, bandwidth, and request-time pressure.

Cloudflare documents R2 as S3-compatible, strongly consistent, and without egress fees. AWS SDK v3 is used because its command-based packages are modular and R2 speaks the S3 API. Compared with local disk, R2 works across replicas and persists across deployments. Compared with AWS S3, R2 is attractive when Cloudflare proximity and egress economics matter; S3 may be better when the broader AWS ecosystem, storage classes, replication, or compliance integration dominates.

Presigned URLs are bearer credentials. The current code trusts client-declared file type and size and creates metadata before upload completion. Production hardening should validate the stored object with HEAD after upload, enforce server-verifiable size/checksum/content type, clean abandoned reservations, scan risky files, and delete R2 objects when metadata is deleted.

3.9 AI SDK, Google provider, VoltAgent, and Zod

The dependency set declares an AI architecture, but the runtime implementation is only a foundation today.

`ai_create_message` converts text plus R2-hosted images, audio, or text files into AI SDK `ModelMessage` structures. Zod defines a small shared tool-query schema. The agent, memory, Elysia server adapter, Google model provider, and VoltAgent packages are installed but not imported by production source.

AI SDK is a provider abstraction for text, structured output, tools, and multimodal messages. It is better than calling one vendor's REST API directly when the team wants consistent message/tool types and future model portability. A direct API client is simpler when only one provider and a few endpoints are needed.

VoltAgent adds agent definitions, tools, memory, workflows, and observability. It becomes valuable when AI behavior needs traces, persistent conversations, reusable tools, multi-step control, or multiple agents. It is unnecessary overhead for a single one-shot model call. `@voltagent/libsql` fits the existing Turso/SQLite direction for memory or trace persistence; `@voltagent/server-elysia` fits the chosen HTTP framework.

Zod is used for AI/tool schemas because VoltAgent and AI libraries commonly consume Zod directly. Elysia's `TypeBox`-based `t` remains better for HTTP routes because it connects runtime validation, TypeScript inference, and OpenAPI in the framework. Maintaining both is reasonable if their boundaries stay clear.

3.10 Docker, Compose, GitHub Actions, Prettier, and custom rules

The production Dockerfile has install, build, and release stages. Dependencies and source stay out of the final stage; only the compiled binary is copied. The process runs as the non-root `bun` user. This lowers image size and attack surface compared with shipping the whole source tree and development toolchain.

The development Dockerfile is actually a compiled development distribution: it also generates Eden output and copies `public`. Compose builds it, loads `.env`, maps port 3000, and restarts always. It does not mount source or run watch mode, so rename it to a preview/integration image or add a true live-development profile.

GitHub Actions installs Bun latest, typechecks, builds Eden declarations, verifies formatting, runs 25 custom rules, and runs tests. Pinning only 'latest' makes the pipeline sensitive to future Bun changes while Docker pins 1.3.14. Use the same version in local tooling, CI, and containers.

Prettier standardizes formatting and sorts imports into architectural groups. The custom rule runner uses the TypeScript compiler API and Bun Glob to enforce file locations, suffixes, exports, naming, path aliases, schema enum independence, and the exact folder tree. This gives more domain-specific control than generic ESLint rules. Biome or ESLint would add broad correctness checks; the custom suite should supplement, not replace, rules for promises, unsafe any, security, and common TypeScript mistakes.

3.11 Direct dependency resolution and actual use

Package	Resolved	Role in current code
elysia	1.4.28	Active: app, controllers, DTO inference, contexts.
@elysiajs/cors / jwt / swagger	1.4.2 / 1.4.2 / 1.3.1	Active: HTTP policy, token handling, development docs.
drizzle-orm / drizzle-kit	0.45.2 / 0.31.10	Active: schemas, queries, generated main migrations.
@libsql/client / @tursodatabase/api	0.17.3 / 1.9.2	Active: SQL connections and tenant DB provisioning.
@aws-sdk/client-s3 / presigner	3.1065.0	Active: R2 client and signed upload URLs.
zod	4.4.3	Partially active: AI/tool utility schema.
ai / @ai-sdk/google	6.0.199 / 3.0.80	Types used indirectly in AI message utility; Google provider not wired.
@voltagent/core / libsql / server-elysia	2.7.7 / 2.1.2 / 2.0.9	Declared and documented, but no production imports yet.
typescript / @types/bun	6.0.3 / 1.3.14	Development type system and Bun API declarations.
prettier / import-sort plugin	3.8.4 / 4.7.1	Development formatting and deterministic import groups.

4. Database and tenancy deep dive

4.1 Main database tables

Table	Purpose	Important fields and constraints
user	Global identity and user-tenant registry	Unique active phone; name/image; schema version; provisioned DB ID/URL; soft delete.
organization	Global organization registry	Name, owner user_id, schema version, DB ID/URL, soft delete.
organization_user	User-to-organization membership	Active pair is unique; indexed in both directions; soft delete.
product	Global normalized product cache	Unique active barcode; type/name/brand; image list; NOVA/Eco/Nutri scores; JSON metadata; update timestamp.
brand	Global normalized brand and boycott snapshot	Unique active normalized name; boolean status; reason and alternative arrays; soft delete.

4.2 Tenant tables

Database type	Tables	Meaning
Every tenant	access, file	Authorization records and R2 object metadata.
Organization	contact plus common	Organization-owned contact records with Morocco-focused enums and arbitrary metadata.
User	scan_history, user_list plus common	Personal scan audit and one active white/black-list state per product.

The schema deliberately avoids importing application enums. SQLite stores generic text or integer values, while DTOs validate allowed values. The rationale is migration flexibility: adding an application enum value should not require rebuilding a SQLite check constraint. The cost is that direct or buggy writes can store invalid values, so every write path and data-repair tool must validate consistently.

No foreign keys are declared. This avoids cross-database constraints, which are impossible between the main database and tenant databases, but even relationships inside the main database are application-enforced. Indexes improve lookup speed but do not guarantee referenced rows exist. Add foreign keys where relationships remain inside one database, or explicitly document why lifecycle and import order make them unsuitable.

Soft deletion is the default retention pattern. Partial unique indexes allow a new active record after an old record is deleted. Every read and mutation must therefore include `deleted_at IS NULL`. The generic select helper does this automatically, but several direct update paths do not, so soft-deleted rows can be touched accidentally.

4.3 Connection routing and LRU cache

- Calling `db_client()` without a tenant returns the cached main Drizzle database.
- Calling it with tenant ID and JWT payload resolves user vs organization, creates the deterministic Turso URL, and returns a tenant Drizzle database.
- Tenant connections are stored in insertion order and refreshed on access, implementing a simple LRU cache. When the configured maximum (default 1800) is reached, the oldest libSQL client is closed and removed.
- There is no idle TTL or telemetry. Tune the limit using real socket/memory measurements and Turso connection behavior, not tenant count alone.
- The fallback tenant type is organization. Safer routing would reject tenant IDs not explicitly present in the authenticated tenant list before any connection is opened.

4.4 Provisioning lifecycle

Step	User tenant	Organization tenant
Registry row	Created during signup	Created by POST /organization
Database creation	Turso API creates db-{app}-{env}-user-{id}	Turso API creates db-{app}-{env}-organization-{id}
Registry update	Stores DB id/url on user	Stores DB id/url on organization
Schema initialization	Syncs common + scan_history + user_list	Syncs common + contact
Owner access	Inserts owner into user tenant access	Inserts owner into organization tenant access
Failure behavior	Throws user-provision-failed after main row exists	Throws organization-provision-failed after registry/membership may exist

Transaction boundary problem

The main database write, Turso control-plane API call, tenant schema creation, and tenant access insert cannot share one ACID transaction. A failure can leave partial state. Model provisioning as a saga with explicit states (pending, provisioning, ready, failed), idempotency keys, retries, reconciliation, and cleanup rather than treating one request stack as atomic.

4.5 Main migrations

Migration	Change	Architectural story
0000_typical_talos	Creates user, organization, organization_user, product and indexes	Initial control-plane and product cache.
0001_shocking_king_bedlam	Renames product company to product brand	Domain vocabulary correction.
0002_handy_lionheart	Creates brand, adds brand_id, removes embedded brand name	Normalizes brand into its own entity.
0003_thick_tarot	Adds brand lookup index	Supports product-to-brand joins.
0004_icy_legion	Adds NOVA, Eco-Score and Nutri-Score columns/indexes	Extends food-product enrichment.
0005_curious_blizzard	Rebuilds score indexes with active-row predicates	Aligns indexes with soft-delete queries.

4.6 Custom tenant schema synchronization

Lazy migration compares PRAGMA table_info and SQLite table names against Drizzle schema objects. It adds missing tables and columns, drops extra columns and tables, drops custom indexes, and recreates target indexes. A Redis NX lock with a 30-second lease prevents two application replicas from migrating the same tenant simultaneously. Schema versions live in the main registry and in the JWT tenant list.

This is a pragmatic answer to fleet migration, but it is the highest-risk subsystem in the repository. The helper compares mostly names, not full type/default/nullability semantics. It treats object defaults as CURRENT_TIMESTAMP heuristically. Several lower-level add/drop helpers catch and log errors without rethrowing, so the caller can mark a schema current even after a failed operation. Destructive drops are automatic, and the process is not wrapped in a transaction or backed up.

- Replace ad hoc diffing with explicit, ordered tenant migration versions. Each version should be idempotent and recorded per tenant.
- Never drop a column/table automatically in the request path. Use expand-migrate-contract releases and a separately approved cleanup job.
- Make failures propagate, preserve the previous version, and expose repair status. Add retry classification and a dead-letter/reconciliation queue.
- Test migrations against databases at every historical version, with realistic data, indexes, nulls, and concurrent requests.
- Back up or snapshot before destructive tenant fleet changes and support canary tenants before broad rollout.

4.7 Generic select engine

The shared select engine implements safe column selection, filters, pagination, count, order, group, soft-delete filtering, and a limited same-column-name left join. Each service supplies an explicit allowed-column map, so clients cannot select arbitrary columns even though they choose their response fields.

Filter operators are encoded as short internal tokens: [] means IN, % prefix match, %% contains-any match, <> between, and standard comparison operators. This keeps services compact but creates a private query language that new contributors must learn.

Important edges: count queries do not replay joins, so a count using a joined-table filter can fail or count incorrectly. Grouped results are counted with raw row count rather than grouped count. Empty results are represented as 404 exceptions rather than successful empty collections, which makes callers use exceptions for normal control flow. Consider returning empty arrays for list endpoints and reserving 404 for singleton resource lookup.

5. Domain modules and business flows

Module	Responsibility	Dependencies	Core idea
auth	Public OTP send/verify, signup/signin, JWT creation	Main user/org membership + Redis OTP	Rate limiting, secure OTP generation, token tenant payload
user	User lookup/create internally; authenticated profile update	Main user + provisioned user DB	Phone uniqueness and tenant creation
tenant	Internal database provisioning and migration	Turso API, main registry, tenant DB, Redis lock	Fleet lifecycle and schema version
organization	User-owned organization CRUD	Main organization and organization_user	Max organizations, membership, DB provisioning
access	Tenant ACL CRUD and authorization	Tenant access + Redis ACL cache	Owner vs full_access semantics
contact	Organization contact CRUD	Organization tenant DB	Tenant ACL and phone uniqueness
file	Metadata CRUD and signed upload reservation	Tenant DB + R2 + Redis limits	Media-only policy and direct upload
brand	Global brand CRUD and boycott snapshot	Main brand	Name normalization and soft delete
product	Global product CRUD and brand join	Main product/brand	Barcode uniqueness, selective projection
product-provider	Open Food Facts lookup and normalization	External API	Timeout, score/tag/image normalization
scan	Barcode/URL/text orchestration	Catalog, providers, history	Cache-aside enrichment and boycott lookup
scan-history	Per-user scan log	User tenant + global product	Cross-database enrichment in application memory
user-list	Per-user white/black list	User tenant + global product	One active row per product
boycott-provider	Boycat search/details adapter	Encrypted external API	AES-256-GCM decode and response normalization
boycott-decision	Internal evidence matching and decision	Curated in-code seed	Aliases, relationships, trust and confidence

5.1 Authentication flow

Send OTP. The controller applies phone and IP limits for short and daily windows. The service verifies whether the requested action is compatible with the user's existence, generates a cryptographically random four-digit code, and stores code plus action in Redis with a configurable TTL. Today it logs the code to the server console; a real SMS provider, delivery status, redacted logs, abuse monitoring, and idempotency are still required.

Verify OTP. Verification checks the Redis value, deletes it after success, then signs in an existing user or creates a new user when the stored action was signup. Signup provisions a user database before returning. Sign-in queries organizations and builds a token containing the user tenant plus all organization tenants and their schema versions.

Authorize requests. The middleware expects a bearer token, verifies it, and derives the payload. JWT tells the app who the user is and which tenant type/version was known at sign-in. Tenant access records remain the authorization source for tenant operations. This distinction is important: authentication is not authorization.

Authentication improvements

Use a production SMS adapter; never log OTP values; add attempt state and lockout telemetry; rotate JWT keys; define refresh/revocation; avoid embedding long-lived membership snapshots; and test proxy IP behavior.

5.2 Organization and tenant ownership

Creating an organization checks a per-user limit, inserts the organization and membership, provisions its database, synchronizes schema, and inserts an owner access record. Find uses the membership join, while update/delete require the original owner user_id. This means members can discover organizations through membership but cannot rename/delete unless they are the owner.

The access service stores actions as JSON. Owners are represented by the string owner; delegated users may have full_access. Mutating access requires owner by default. Redis caches an authenticated user's action array for five hours and is invalidated by create/update/delete.

There is a contract mismatch: the database and internal service can store owner, but the public access DTO only allows `full_access` in returned action arrays. An owner row returned by GET /access may fail response validation. Make owner a first-class enum value or separate role from permissions.

5.3 Contacts

Contacts live only in organization tenant databases. The DTO supports phone, name, gender, birthday, national/passport IDs, address, Morocco city/country/nationality, status, and arbitrary JSON metadata. The service checks tenant access, enforces active phone uniqueness through a find-then-create flow, and soft-deletes records.

This module shows the intended reusable pattern most clearly: thin controller, comprehensive DTO, service authorization, generic select, and tenant-routed database. For a new contributor, it is the best first module to trace end to end.

5.4 Files and object storage

The client asks for an upload reservation with tenant ID, file name, MIME type, and declared size. The service checks access, per-minute and daily user limits, allows only image/audio/video prefixes, checks category-specific maximums, generates a random user-scoped key, signs a PutObject command, inserts metadata, and returns the URL.

Update only changes the display name for metadata owned by the current user. Delete soft-deletes metadata but does not delete the R2 object. Find supports selective columns and prefix search. There is no download-signing endpoint; AI utilities instead fetch through the configured R2 link.

The direct-upload pattern is efficient, but validation is advisory until the backend inspects the actual stored object. Use a finalize endpoint or R2 event, verify content length/checksum/type, scan files, and transition metadata from reserved to ready. A cleanup job should remove expired reservations and orphaned objects.

5.5 Product catalog and Open Food Facts

Products are global and identified by numeric barcode. They may contain a normalized brand link, image URLs, NOVA group, Eco-Score, Nutri-Score, ingredients, and allergens. Product find can include joined brand boycott fields. Brand names are trimmed and lowercased before creation, providing a simple canonical form.

The Open Food Facts adapter requires a User-Agent, applies a configurable abort timeout, fetches API v2, rejects unsuccessful or absent products, deduplicates image URLs, normalizes score grades, validates NOVA 1-4, strips language prefixes from ingredient/allergen tags, and returns a provider-neutral DTO. Provider errors and timeouts collapse to null, which the scan service presents as not found.

The cache-aside design reduces repeated provider calls and stabilizes product response shape. It needs cache freshness policy: products can change upstream, but no refresh TTL or revalidation exists. Add provider timestamps, stale-after policy, background refresh, and provenance per field.

5.6 Scan orchestration

A numeric value is a barcode. A URL is inspected for barcode-like query parameters, path, or hash. Other input becomes text search. Barcode resolution first queries the global catalog. If absent, it fetches Open Food Facts, creates/finds a brand, stores the product, and returns `source=provider`; otherwise `source=cache`.

Every successful barcode scan writes to the authenticated user's tenant database. Because product truth is global and history is tenant-private, scan-history reads perform an application-level join: load history from the user DB, collect unique product IDs, fetch global products in one batch, then merge using a map. User lists use the same pattern.

Boycott resolution first asks Boycat directly using brand or product name. If no direct match, it builds up to six canonical candidates from comma/semicolon/pipe-separated brands and product name, searches, picks the best result, and asks for details again. A matched provider decision updates the global brand's boycott snapshot. This makes later product reads fast, but a third-party result can mutate global data for all users; provenance and freshness must be stored alongside the snapshot.

5.7 Two boycott decision paths

Dimension	Boycat provider path	Internal knowledge path
Endpoint	/boycott-provider/search and /decide; used by scan	/boycott-decision/decide; not used by scan
Data	Live encrypted provider response	Nine in-code knowledge items from BDS, OpenSanctions, TechForPalestine, and manual seed
Matching	Exact normalized name gets 100; otherwise 80; search fallback	Exact, alias, website, Dice fuzzy, and related-entity paths
Confidence	Campaign tier maps to 70-100	45% match + 30% source trust + 25% item confidence + tier bonus
Failure	Returns unavailable/invalid_response with unknown decision	Returns unknown when no reliable match
Strength	Fresh provider campaign detail and alternatives	Transparent, testable, multi-source evidence model
Weakness	External dependency, key/decryption contract, opaque freshness	Small static seed that can become stale

The internal matcher removes accents, punctuation, and corporate suffixes; compares canonical names and aliases; extracts domains; and uses bigram Dice similarity for fuzzy matches. Related entities carry ownership or identity relationships into the matched path. Low scores under 55 return unknown. Active boycott/risk claims can become boycott or needs_review; matched inactive neutral claims can become not_boycotted.

The two engines should converge behind one decision interface. A mature design would gather evidence from both providers, version the evidence snapshot, deduplicate entities, compute policy-specific decisions, and retain the exact inputs and sources used. The wording correctly warns that not_boycotted means no active evidence in current data, not certification of safety.

Provider URL concern

The Boycat detail URL first applies encodeURIComponent and then assigns the result through URLSearchParams, producing double encoding such as Coca%2520Cola. The test asserts that behavior, but it should be verified against the real provider contract and corrected if the API expects normal query encoding.

5.8 AI foundation

The AI utility can build one user ModelMessage containing text plus images, audio files, or text-file contents fetched from R2. MIME type is inferred from extension. Unsupported files are skipped with a warning. This is a useful adapter boundary because agents can receive model-ready messages without knowing storage details.

There is currently no src/ai/agent implementation, no tool-specific folder, no VoltAgent server composition, no model invocation, and no persistent memory. The .agents examples define the intended conventions for future agents, prompts, schemas, tools, and toolkits. Treat installed AI packages as planned capability, not a completed feature.

Before enabling AI, add fetch status and timeout checks, object-size limits before buffering, private authenticated object retrieval, prompt-injection boundaries for text files, model cost limits, redaction, tool authorization tied to real user identity, and evaluation datasets. The current tool context returns a synthetic user_id of -1, which is not a production authorization model.

6. HTTP API inventory

All routes after the authentication controller are inside the JWT-derived protected group. Swagger adds a development-only documentation route. The table describes intent; consult DTOs for exact field-level constraints.

Method	Path	Guard	Purpose
GET	/	Public	HTML service banner
GET	/eden	Dev only	Generated TypeScript declaration
GET	/swagger	Dev only	Interactive OpenAPI documentation supplied by plugin
POST	/auth/otp/send	Public + limits	Validate sign-in/up intent and create OTP
POST	/auth/otp/verify	Public + limits	Verify OTP, sign in or provision signup, return JWT
PATCH	/user	JWT	Update authenticated profile
GET	/organization	JWT	Find organizations available through membership
POST	/organization	JWT	Create and provision organization tenant
PATCH	/organization	JWT owner	Rename owned organization
DELETE	/organization	JWT owner	Soft-delete owned organization registry
GET	/brand	JWT	Find global brands
POST	/brand	JWT	Create global normalized brand
PUT	/brand	JWT	Update global brand and boycott snapshot
DELETE	/brand	JWT	Soft-delete global brand
GET	/product	JWT	Find global products with optional brand projection
POST	/product	JWT	Create global product
PATCH	/product	JWT	Update global product
DELETE	/product	JWT	Soft-delete global product
POST	/scan/barcode	JWT + user tenant	Resolve barcode and record scan history
POST	/scan/identify	JWT + user tenant	Interpret barcode/URL/text and resolve product/boycott
POST	/boycott-provider/search	JWT	Search Boycat brand candidates
POST	/boycott-provider/decide	JWT	Get normalized Boycat brand decision
POST	/boycott-decision/decide	JWT	Run internal evidence matcher
GET	/scan-history	JWT + user tenant	Read user history enriched with products
GET	/user-list	JWT + user tenant	Read personal white/black list
POST	/user-list	JWT + user tenant	Add product to one list
PATCH	/user-list	JWT + user tenant	Switch list type
DELETE	/user-list	JWT + user tenant	Soft-delete list entry
GET	/file	JWT + tenant owner	Find tenant file metadata
POST	/file	JWT + tenant owner + limits	Reserve metadata and return signed R2 PUT
PATCH	/file	JWT + tenant owner	Rename own file metadata
DELETE	/file	JWT + tenant owner	Soft-delete own file metadata
GET	/contact	JWT + org owner	Find organization contacts
POST	/contact	JWT + org owner	Create contact
PATCH	/contact	JWT + org owner	Update contact
DELETE	/contact	JWT + org owner	Soft-delete contact
GET	/access	JWT owner	Find tenant access records
POST	/access	JWT owner	Grant tenant actions
PATCH	/access	JWT owner	Replace tenant actions
DELETE	/access	JWT owner	Revoke tenant access

6.1 Query contract

- Find routes require a columns array, which makes sparse responses explicit and can reduce payload size.

- Common query fields include page, take, count, combination_type, order_by, and group_by. Maximum take defaults to 12 and is environment-configurable.
- Descending order uses a leading hyphen. Service-specific column enums prevent arbitrary ordering and projection.
- Filters are arrays for IN/prefix operators. Numeric query values use Elysia Numeric so string query parameters can become numbers.
- An empty collection currently throws a table-specific not-found error. Decide whether client ergonomics favor HTTP 200 with an empty data array instead.

7. Configuration, build, and delivery

7.1 Environment variables

Group	Variables	Purpose
Application	APP_NAME, ENV, PORT, REQUEST_TIMEOUT, QUERY_TAKE_MAX	Identity, environment switch, HTTP behavior, pagination ceiling
Authentication	JWT_SECRET_KEY, OTP_TTL_MINUTES	Signing and OTP lifetime
Turso control plane	TURSO_API_TOKEN, TURSO_ORG_NAME, TURSO_GROUP_NAME, TURSO_GROUP_TOKEN	Create and access tenant databases
Main database	TURSO_DB_MAIN_URL, TURSO_DB_MAIN_TOKEN	Global database connection
Tenant connections	DB_MAX_CACHED_TENANTS	LRU capacity
Redis	REDIS_DB_AUTH_URL, REDIS_DB_TENANT_ACCESS_URL, REDIS_DB_MIGRATION_LOCK_URL, REDIS_DB_RATE_LIMITING_URL	Four coordination clients
R2	CLOUDFLARE_ACCOUNT_ID, R2 access key/secret, bucket, link, signed URL expiry	Object storage access and public/private fetch base
File policy	MAX_IMAGE_SIZE_BYTES, MAX_AUDIO_SIZE_BYTES, MAX_VIDEO_SIZE_BYTES	Declared-size limits
Global rate limit	RATE_LIMIT_GLOBAL_IP_LIMIT, RATE_LIMIT_GLOBAL_IP_DURATION	All-request fixed window
Upload limits	RATE_LIMIT_FILE_UPLOAD_LIMIT/DURATION and DAILY variants	Per-user short/daily upload reservations
OTP send limits	Phone/IP short and daily limit/duration pairs	Anti-abuse across two dimensions
OTP verify limits	Short and daily verify limit/duration pairs	Brute-force protection
Business limits	USER_MAX_ORGANIZATIONS	Owner organization ceiling
Open Food Facts	OPEN_FOOD_FACTS_USER_AGENT, BASE_URL, TIMEOUT_MS	Provider identity, endpoint, abort
Boycat optional	BOYCAT_COMPLIANCE_URL, BOYCAT_API_KEY, BOYCAT_TIMEOUT_MS	Encrypted provider; present in example but intentionally not startup-required
AI	GOOGLE_GENERATIVE_AI_API_KEY	Required at startup even though the Google provider is not yet used

Configuration drift

GOOGLE_GENERATIVE_AI_API_KEY is required although no production model call exists, while optional Boycat variables drive a live feature. Split required variables by enabled feature and environment so unused integrations do not block boot and enabled integrations cannot silently degrade without an explicit readiness signal.

7.2 Package scripts

Script	What it does	When to use
dev	Loads .env and watches src/app.ts	Local API development
build	Compiles/minifies a Bun-targeted binary	Artifact build
start	Builds then executes binary	Simple direct deployment; inefficient if rebuilding every start
typecheck	Runs tsc without emit	Static correctness gate
format / format:check	Writes or verifies Prettier output	Local cleanup / CI
rules	Runs 25 project-specific checks	Architecture/naming gate
test	Runs Bun tests under src	Unit/service behavior
eden	Emits app declaration and copies to public/eden.ts	Frontend contract sharing
all	Format, typecheck, rules, tests, Eden	Pre-review local gate; note that it mutates formatting/output
db:generate / migrate / studio	Drizzle main DB migration workflow and browser	Schema development/operations

7.3 TypeScript and formatting philosophy

- ESNEXT target/lib and preserved modules let Bun/bundler handle modern syntax rather than downleveling for old runtimes.
- Bundler resolution and TypeScript extension imports align with Bun's execution model.
- Strict and noUncheckedIndexedAccess force developers to handle missing values. The code still uses non-null assertions and any at integration boundaries, so tests and runtime schemas remain essential.

- Prettier uses two spaces, no semicolons, single quotes, trailing commas, 150-column width, and deterministic import sections.
- The project documentation forbids comments. That encourages readable naming but can be counterproductive for security invariants, migration reasoning, and external protocol quirks. Prefer concise rationale comments where the 'why' cannot be encoded in names or tests.

7.4 CI and deployment

CI runs on pushes and pull requests to main. It checks types, Eden generation, formatting, architecture rules, and functional tests. CODEOWNERS assigns every path to one owner. The pipeline does not compile the production binary, build the Docker image, boot the app, migrate a database, or test against Redis/R2/provider contracts.

The production image installs with a frozen lockfile, compiles, copies only the binary, exposes port 3000, and runs non-root. It should add an immutable base-image digest, vulnerability scanning, SBOM/provenance, healthcheck, read-only filesystem where feasible, and a deployment-time smoke test. Separate build and run in the start script so containers do not rebuild on boot.

The README is stale: it still says elysia-backend and instructs `bun run index.ts`, but the entry point is `src/app.ts`. Replace it with architecture, prerequisites, environment setup, commands, migrations, tenant lifecycle, provider setup, test strategy, and troubleshooting.

8. Code-quality system and tests

8.1 Executable rules

Concern	Enforcement	Value
Naming	Lowercase/kebab paths; snake_case variables/functions; PascalCase classes	Consistent searchability and predictable generated names
Module exports	controller_x, service_x, dto_x; no default exports	Stable composition and discoverability
File suffixes	.controller/.service/.dto/.test plus provider/util/types/seed	File role is visible in its name
Location	Module/lib/db/test/agent/tool files must live in approved roots	Prevents architectural erosion
Imports	No relative imports in src	Alias communicates ownership
DB schema	No application enum imports	Avoid enum-driven SQLite migration coupling
Root tree	Only approved dirs/files	Makes project shape an invariant
AI structure	Agent, prompt, schema, tool suffix/location patterns	Future AI code remains modular

The rule runner dynamically imports module files to inspect named exports and parses TypeScript ASTs for names/imports. That is stronger than filename-only shell scripts, but dynamic imports can execute module initialization. Keep rule-loaded files free of network connections and side effects, or change these checks to inspect AST exports instead.

Some documented rules and actual code diverge. The contributor rules say database access must go through services, while services directly use Drizzle by design. The example database skill contains an enum import despite a prominent rule forbidding it. Treat examples as versioned code: test them or delete stale examples so contributors do not receive contradictory guidance.

8.2 Test coverage shape

Area	Tests	What is covered
Auth	2	Sign-in token and OTP send success
Tenant	7	User/org provisioning, migration, current version, missing URL, lock contention
Organization/user	9	CRUD/provisioning limit/failure and profile flows
Catalog/providers	18	Brand/product CRUD, Open Food Facts mapping, Boycat encrypted responses
Scan/decision	12	Cache/provider resolution, URL/text identification, matching and confidence states
Tenant features	19	Access cache/roles, contact/file CRUD
User features	6	History enrichment and list CRUD
Total	71	Primarily service unit tests with mocked databases, Redis, storage, fetch, and provisioning

The tests are fast and focused because Bun module mocks replace infrastructure. This is useful for business branching, but mock objects often reproduce only the chain needed by the test. They cannot prove generated SQL, response validation, lifecycle scope, database constraints, Redis atomicity, R2 signatures, or real provider compatibility.

Add a test pyramid: pure unit tests for matching/query builders; service integration tests using local SQLite/libSQL and Redis; Elysia HTTP contract tests through Eden/fetch; provider contract fixtures; migration tests across historical schemas; and a small end-to-end path that boots the compiled container with disposable infrastructure.

Observed test drift

One tenant test uses schema version 0.0.6 while `current_schema_version` is 0.0.1. Because the mock flow still returns true, the test name may not prove the intended skip branch. Use the exported current version in the fixture and assert that `sync_schema` was not called.

9. Security, reliability, and production-readiness review

Priorities below are recommendations from static analysis, not confirmed incidents. 'Critical' means a defect could cause cross-tenant impact or destructive data behavior if the path is exercised in production.

Priority	Finding	Impact	Recommended direction
Critical	Tenant schema sync can drop data and swallow lower-level errors	Partial or destructive tenant migrations may be marked current	Versioned non-destructive migrations, error propagation, canaries, backups, repair queue
Critical	Any authenticated user can mutate global product/brand catalog	One user can affect all users' catalog and boycott snapshot	Admin/service roles; separate ingestion endpoints; audit log; provenance
High	Provisioning spans multiple systems without durable state	Orphan registry rows, databases, or missing owner ACL	Saga/status state, idempotent worker, reconciliation and cleanup
High	Upload type/size are client claims; metadata precedes upload	Oversized/malicious/orphan objects and false metadata	Finalize/HEAD verification, checksum, scanning, reservation expiry
High	File deletion does not remove object	Unbounded storage, privacy/retention failure	DeleteObject workflow and retryable cleanup log
High	JWT membership/version snapshot has no revocation policy	Stale membership and long-lived authorization assumptions	Shorter access token, refresh/session store, revocation/version checks
High	Owner value is absent from access DTO enum	Response validation or client contract failure	Model roles and permissions consistently
High	Generic count ignores joins	Organization count queries can fail or miscount	Build count from same joined query/filter plan
Medium	Soft-delete predicates missing on some mutations	Old and active rows can be updated together	Include isNull in every mutation; add repository helpers/tests
Medium	OTP is logged and no SMS provider exists	Credential exposure and incomplete authentication	Secure provider, redaction, delivery state, alerting
Medium	Error mapper may expose raw error.message	Internal details can leak to clients	Public-safe messages; structured private logs with correlation IDs
Medium	No explicit production CORS origin allowlist	Unexpected cross-origin access behavior	Environment-specific exact origins and headers
Medium	Provider failures collapse into not found	Operational outage looks like missing product	Differentiate not_found, timeout, unavailable, invalid_response
Medium	Brand/product create use check-then-insert	Concurrent unique conflicts become generic failures	Atomic upsert or catch/map unique constraint
Medium	No health/readiness or telemetry	Slow detection and unsafe rollout	Health, dependency readiness, metrics, traces, structured logs
Low	README, lockfile name, CI/runtime versions drift	Onboarding and reproducibility friction	Single version source and updated docs

9.1 Security mechanisms already present

- Fail-fast environment validation and secret variables excluded from Git and Docker context.
- Cryptographically random OTP and file-key components rather than Math.random.
- JWT verification before protected groups and tenant ACL checks inside tenant services.
- Global, OTP-specific, and upload-specific Redis rate limits.
- Security headers including HSTS, CSP, frame denial, nosniff, referrer and permissions policies.
- Soft deletion and partial unique indexes for active identities/catalog entries.
- Non-root container runtime, frozen dependency installation, and minimal release artifact.
- AES-256-GCM authenticated decryption for Boycat provider payloads.

9.2 Observability design to add

Use structured JSON logs with request ID, user ID (appropriately protected), tenant type/ID, route, status, latency, provider status, database target class, and error code. Never log JWTs, OTPs, API keys, signed URLs, contact identifiers, or full provider payloads by default.

Metrics should include request rate/errors/latency; Redis errors and limiter rejections; tenant connection cache size/evictions; migration attempts/duration/failures/version lag; provisioning states; provider latency/status; scan cache hit rate; R2 reservations/finalization/orphans; and decision status/confidence distribution.

Traces are particularly valuable for scan and provisioning because one request crosses several services. OpenTelemetry or VoltAgent observability can cover different domains: application traces for HTTP/data/provider flows, VoltAgent traces for future model/tool decisions.

9.3 Production rollout order

Order	Outcome	Deliverables
1	Make data changes safe	Replace tenant sync drops, add provisioning state/reconciliation, object cleanup
2	Close authorization gaps	Admin roles for global catalog, JWT/session policy, owner contract
3	Make uploads verifiable	Finalize, HEAD/checksum/type/size, malware policy, reservation cleanup
4	Make failures visible	Health/readiness, structured logs, metrics, traces, alerts
5	Prove with integration tests	Live libSQL/Redis, HTTP contracts, migrations, container boot
6	Align delivery	Pinned Bun across CI/Docker, build image in CI, update README/lock
7	Scale deliberately	Load test connection cache, rate limits, provider concurrency, cross-tenant operations
8	Activate AI safely	Identity-aware tools, evaluations, cost/redaction limits, memory/trace policy

10. Tailored learning roadmap

The fastest path is not to study every library independently. Learn one vertical slice at a time, then widen into infrastructure and distributed-system concepts. Each phase below includes a concrete proof of understanding.

Time	Theme	Concepts	Proof of understanding
Phase 1: 2-3 days	TypeScript and Bun	strict types, unions, generics, async/await, ESM, Bun scripts/test/build	Explain lib_dto_context and fix one any cast without changing behavior
Phase 2: 3-4 days	HTTP and Elysia	plugins, lifecycle, context derivation, schema validation, error mapping, Eden	Trace PATCH /user from bytes to response; add an HTTP contract test
Phase 3: 4-5 days	SQL, SQLite/libSQL, Drizzle	tables/indexes, partial unique indexes, joins, transactions, query plans, migrations	Write SQL equivalents for product.find and inspect EXPLAIN plans
Phase 4: 4-5 days	Tenancy and authorization	control/data planes, isolation models, JWT vs ACL, cache invalidation	Draw user/org provisioning and list every partial-failure state
Phase 5: 3-4 days	Redis/distributed coordination	TTL, atomicity, locks, fixed/sliding windows, fail-open/closed	Implement/test an atomic limiter and migration lock renewal design
Phase 6: 3-4 days	Object storage	S3 commands, signatures, checksums, metadata lifecycle, malware scanning	Design upload reservation -> finalize -> cleanup state machine
Phase 7: 4-5 days	External integrations	timeouts, retries, circuit breakers, normalization, provenance, contract tests	Split OFF not-found vs unavailable and add fixtures
Phase 8: 4-5 days	Testing and delivery	unit/integration/e2e, Docker layers, CI reproducibility, observability	Boot an ephemeral stack and test signup -> scan -> history
Phase 9: 5-7 days	Evidence and AI	entity resolution, confidence, structured output, tools, memory, evaluations	Create a golden dataset and compare both boycott engines

10.1 Concepts to go deep on

Concept	Study questions
Type systems	Discriminated unions, structural typing, inference, variance basics, branded IDs, unknown vs any, runtime schema alignment.
HTTP	Idempotency, method semantics, status codes, content negotiation, CORS, proxies, timeouts, cancellation, retry safety.
SQL	Normalization, constraints, indexes, selectivity, partial indexes, transactions, isolation, query plans, soft-delete tradeoffs.
Multi-tenancy	Shared-table RLS, schema-per-tenant, database-per-tenant, noisy neighbor, residency, backups, tenant moves, analytics.
Distributed systems	At-least-once work, idempotency, leases, fencing tokens, sagas, outbox, reconciliation, circuit breakers.
Security	Threat modeling, authentication vs authorization, secret rotation, least privilege, audit logs, upload security, privacy retention.
Caching	Cache-aside, invalidation, stampede, TTL, stale-while-revalidate, negative caching, provenance/freshness.
Evidence systems	Entity resolution, source trust, temporal validity, explainability, conflicting evidence, human review, policy versioning.
AI agents	Prompt/tool boundaries, structured output, model evaluation, tracing, memory scopes, injection, cost/latency controls.
Operations	SLOs, RED/USE metrics, health vs readiness, deployment strategies, migrations, backups, incident response.

10.2 Recommended reading order inside this repository

- 1. package.json, tsconfig.json, .prettierrc, .env.example - learn the tool and configuration contract.
- 2. src/app.ts, src/lib/env.lib.ts, jwt.lib.ts, middleware.lib.ts, error.lib.ts - learn startup and request lifecycle.
- 3. src/module/organization/contact - learn the standard controller/DTO/service slice.
- 4. src/db/main.schema.db.ts and utils.db.ts - learn table definitions and the generic query engine.
- 5. src/db/client.db.ts plus tenant/user/organization schemas - learn database routing.
- 6. main/tenant, main/organization, tenant/access - learn provisioning, schema versions, and authorization.
- 7. main/product, main/brand, main/product-provider - learn catalog normalization and provider adapters.
- 8. main/scan plus user/scan-history and user/user-list - learn cross-module orchestration and application joins.
- 9. boycott-provider and boycott-decision - compare live adapter normalization with internal evidence reasoning.
- 10. ai utilities and .agents examples - understand planned agent/tool structure after the normal service architecture is clear.
- 11. rule directory - learn how architectural intent is enforced.
- 12. tests, migrations, Docker, CI - verify behavior and delivery assumptions last, with the domain model already understood.

10.3 Practical exercises

Level	Exercise	What it teaches
Beginner	Add GET /user/me	Reuse authenticated payload, service lookup, DTO response, controller pattern, and tests.
Beginner	Improve README	Document real setup, commands, architecture, environment groups, and troubleshooting.
Intermediate	Return empty collections as 200	Change generic select semantics and update callers/tests consistently.
Intermediate	Finalize R2 upload	Add reserved/ready status, HEAD verification, checksum, expiry, and cleanup.
Intermediate	Fix joined counts	Build count from the same join/filter plan and test organization membership counts.
Advanced	Provisioning saga	State machine, idempotency, worker, retries, compensation, admin repair API.
Advanced	Versioned tenant migrations	Historical fixtures, expand/contract, fleet status, canaries, rollback/restore.
Advanced	Unified boycott decision	Provider interface, evidence store, temporal provenance, policy version, human review.
Advanced	Production observability	Correlated HTTP/provider/DB traces, RED metrics, dashboards, alerts and SLOs.
AI	First safe agent	Read-only product/decision tools, real identity context, Zod schemas, evaluations, traces and budgets.

11. Project-management view

11.1 Suggested workstreams

Workstream	Scope	Definition of progress
Data platform	Tenant migrations, provisioning, backups, reconciliation	Zero untracked partial tenants; migration success and lag SLO
Identity/security	OTP provider, sessions/JWT, roles, audit, CORS	No global mutation without role; revocation and key rotation tested
Media	Verified uploads, lifecycle, cleanup, privacy deletion	Every metadata row maps to one verified object; orphan rate near zero
Catalog/scan	Freshness, provenance, provider resilience, cache policy	Measured cache hit, provider availability, stale-data rate
Decision quality	Unified evidence, review workflow, golden evaluations	Precision/recall and explanation quality tracked by dataset version
Developer platform	Pinned toolchain, integration tests, CI image, docs	One-command reproducible setup and green boot smoke test
Operations	Health, logs, metrics, traces, dashboards, runbooks	Defined SLOs and actionable alerts with owner
AI	Agent/tool implementation after safety foundation	Read-only evaluated agent before mutation tools

11.2 A realistic delivery sequence

Milestone A - reproducible baseline: align Bun versions, regenerate the lockfile, update README, make CI build the production image, and add a boot smoke test with feature-aware environment validation.

Milestone B - safe data lifecycle: introduce tenant provisioning states and reconciliation, disable destructive lazy schema drops, add explicit tenant migrations, and verify backup/restore on a sample tenant fleet.

Milestone C - secure public beta: integrate SMS, implement token/session lifecycle, add admin roles for global catalog changes, verify uploads, implement R2 deletion, configure CORS/proxy trust, and add audit logs.

Milestone D - observable beta: add health/readiness, structured logs, metrics, traces, dashboards, provider circuit breakers, and on-call runbooks. Load-test tenant connections and scan orchestration.

Milestone E - decision quality: unify the two boycott paths, store versioned provenance, build a review queue, and measure decisions against a curated multilingual dataset.

Milestone F - AI capability: add read-only tools first, persistent memory only where justified, model/provider evaluation, cost and latency budgets, prompt-injection controls, and trace review before any write-capable agent.

11.3 Definition of done for backend changes

- DTO defines and validates input/output, including failure response behavior.
- Authorization decision is explicit and tested for denied as well as allowed users.
- Database change has an expand/migrate/contract plan, historical fixture test, and rollback/repair path.
- External call has timeout, cancellation, classified errors, retry/idempotency decision, and contract fixture.
- Logs and metrics expose success, failure, latency, and dependency status without secrets or sensitive personal data.
- Unit, integration, and HTTP contract tests cover the risk; mocks do not stand in for the only proof.
- CI, container build, documentation, and example environment are updated together.
- Operational owner, alert, cleanup/reconciliation, and privacy retention are defined when state crosses systems.

12. File-by-file repository appendix

This appendix accounts for every tracked project file by path and role. Generated files are described as generated artifacts rather than repeated verbatim. The inventory excludes .git internals and the PDF output itself.

Path	Bytes	Lines	Role
.agents/rules/code-standards.md	1,701	28	Always-on contributor/agent rule
.agents/rules/general-rules.md	1,202	35	Always-on contributor/agent rule
.agents/rules/role-and-mission.md	455	13	Always-on contributor/agent rule
.agents/skills/application-structure/SKILL.md	1,620	53	Architecture or implementation pattern skill
.agents/skills/architecture/SKILL.md	382	8	Architecture or implementation pattern skill
.agents/skills/example-agent/SKILL.md	1,500	45	Architecture or implementation pattern skill
.agents/skills/example-controller/SKILL.md	908	38	Architecture or implementation pattern skill
.agents/skills/example-database/SKILL.md	5,392	154	Architecture or implementation pattern skill
.agents/skills/example-dto/SKILL.md	1,828	72	Architecture or implementation pattern skill
.agents/skills/example-errors/SKILL.md	901	24	Architecture or implementation pattern skill
.agents/skills/example-file-service/SKILL.md	2,031	76	Architecture or implementation pattern skill
.agents/skills/example-service/SKILL.md	738	23	Architecture or implementation pattern skill
.agents/skills/example-shared-reusable-dto/SKILL.md	948	42	Architecture or implementation pattern skill
.agents/skills/example-shared-reusable-enum/SKILL.md	240	10	Architecture or implementation pattern skill
.agents/skills/example-tool/SKILL.md	1,848	56	Architecture or implementation pattern skill
.agents/skills/tech-stack/SKILL.md	968	33	Architecture or implementation pattern skill
.dockerignore	82	9	Container build-context exclusions
.drizzle/0000_typical_talos.sql	3,067	60	Versioned main-database SQL migration
.drizzle/0001_shocking_king_bedlam.sql	83	1	Versioned main-database SQL migration
.drizzle/0002_handy_lionheart.sql	809	15	Versioned main-database SQL migration
.drizzle/0003_thick_tarot.sql	87	1	Versioned main-database SQL migration
.drizzle/0004_icy_legion.sql	500	6	Versioned main-database SQL migration
.drizzle/0005_curious_bizzard.sql	524	6	Versioned main-database SQL migration
.drizzle/meta/0000_snapshot.json	10,805	410	Generated Drizzle schema snapshot
.drizzle/meta/0001_snapshot.json	10,889	412	Generated Drizzle schema snapshot
.drizzle/meta/0002_snapshot.json	13,100	497	Generated Drizzle schema snapshot
.drizzle/meta/0003_snapshot.json	13,315	505	Generated Drizzle schema snapshot
.drizzle/meta/0004_snapshot.json	14,475	547	Generated Drizzle schema snapshot
.drizzle/meta/0005_snapshot.json	14,598	550	Generated Drizzle schema snapshot
.drizzle/meta/_journal.json	915	48	Generated migration journal
.env.example	1,390	68	Environment-variable contract without secrets
.github/CODEOWNERS	11	1	Repository ownership rule
.github/workflows/ci.yml	721	41	GitHub Actions continuous-integration workflow
.gitignore	429	39	Git exclusions
.prettiignore	87	10	Formatting exclusions
.prettierrc	647	36	Formatting and import-order configuration
bun.lock	234,710	7,608	Generated exact dependency resolution lockfile
docker-compose.yml	195	11	Local container orchestration
Dockerfile	468	23	Production multi-stage container build
Dockerfile.dev	526	25	Compiled development/Eden container build
drizzle.config.ts	308	12	Drizzle Kit main database configuration
eden.types.json	256	12	Declaration-only Eden type build configuration

Path	Bytes	Lines	Role
package.json	1,666	45	Package metadata, scripts, direct dependencies
README.md	230	15	Project entry documentation (currently stale)
rule/index.rule.ts	5,407	92	Custom rule registry and runner
rule/lib/agent-ts-location-violations.rule.ts	641	19	Custom codebase rule implementation
rule/lib/ai-agent-extension-violations.rule.ts	958	25	Custom codebase rule implementation
rule/lib/ai-root-file-extension-violations.rule.ts	755	23	Custom codebase rule implementation
rule/lib/ai-tool-extension-violations.rule.ts	868	24	Custom codebase rule implementation
rule/lib/class-naming-violations.rule.ts	818	25	Custom codebase rule implementation
rule/lib/controller-violations.rule.ts	898	23	Custom codebase rule implementation
rule/lib/db-extension-violations.rule.ts	867	24	Custom codebase rule implementation
rule/lib/db-no-enum-imports.rule.ts	1,024	26	Custom codebase rule implementation
rule/lib/db-ts-location-violations.rule.ts	594	18	Custom codebase rule implementation
rule/lib/dto-violations.rule.ts	877	23	Custom codebase rule implementation
rule/lib/entry-point-violations.rule.ts	733	23	Custom codebase rule implementation
rule/lib/folder-structure-violations.rule.ts	2,451	70	Custom codebase rule implementation
rule/lib/function-naming-violations.rule.ts	840	25	Custom codebase rule implementation
rule/lib/kebab-case-violations.rule.ts	749	23	Custom codebase rule implementation
rule/lib/lib-extension-violations.rule.ts	754	23	Custom codebase rule implementation
rule/lib/lib-ts-location-violations.rule.ts	458	15	Custom codebase rule implementation
rule/lib/lowercase-violations.rule.ts	671	23	Custom codebase rule implementation
rule/lib/module-file-location-violations.rule.ts	507	15	Custom codebase rule implementation
rule/lib/module-file-type-violations.rule.ts	1,041	27	Custom codebase rule implementation
rule/lib/relative-import-violations.rule.ts	824	23	Custom codebase rule implementation
rule/lib/service-violations.rule.ts	889	23	Custom codebase rule implementation
rule/lib/test-ts-location-violations.rule.ts	460	15	Custom codebase rule implementation
rule/lib/tool-ts-location-violations.rule.ts	577	18	Custom codebase rule implementation
rule/lib/ts-extension-violations.rule.ts	755	23	Custom codebase rule implementation
rule/lib/variable-naming-violations.rule.ts	857	25	Custom codebase rule implementation
rule/utlis.rule.ts	1,551	49	Custom rule parsing/logging utilities
src/ai/tool/utlis.tool.ts	535	17	AI/tool foundation utility
src/ai/utlis.ai.ts	2,828	97	AI/tool foundation utility
src/app.ts	3,707	127	Application composition root and lifecycle
src/db/client.db.ts	3,686	123	Database connection/query/migration utility
src/db/main.schema.db.ts	5,547	149	Drizzle SQLite/libSQL table schema
src/db/organization.schema.db.ts	1,432	34	Drizzle SQLite/libSQL table schema
src/db/tenant.schema.db.ts	1,135	38	Drizzle SQLite/libSQL table schema
src/db/user.schema.db.ts	1,482	43	Drizzle SQLite/libSQL table schema
src/db/utlis.db.ts	16,937	441	Database connection/query/migration utility
src/db/utlis.dto.db.ts	933	33	Database utility types
src/lib/dto.lib.ts	1,010	44	Cross-cutting shared library
src/lib/enum.lib.ts	2,096	82	Cross-cutting shared library
src/lib/env.lib.ts	1,766	59	Cross-cutting shared library
src/lib/error.lib.ts	4,073	90	Cross-cutting shared library
src/lib/jwt.lib.ts	117	6	Cross-cutting shared library
src/lib/middleware.lib.ts	4,055	109	Cross-cutting shared library
src/module/main/auth/auth.controller.ts	2,776	59	HTTP controller / Elysia plugin

Path	Bytes	Lines	Role
src/module/main/auth/auth.dto.ts	785	30	Runtime and TypeScript API contract
src/module/main/auth/auth.service.ts	4,909	133	Business logic service
src/module/main/auth/auth.test.ts	2,492	91	Bun unit/service test
src/module/main/boycott-decision/boycott-decision.controller.ts	549	20	HTTP controller / Elysia plugin
src/module/main/boycott-decision/boycott-decision.dto.ts	3,763	108	Runtime and TypeScript API contract
src/module/main/boycott-decision/boycott-decision.seed.ts	10,976	324	Curated in-code knowledge seed
src/module/main/boycott-decision/boycott-decision.service.ts	3,410	92	Business logic service
src/module/main/boycott-decision/boycott-decision.test.ts	2,149	61	Bun unit/service test
src/module/main/boycott-decision/provider/boycott-knowledge.provider.ts	332	9	External or internal provider adapter
src/module/main/boycott-decision/source/source.types.ts	330	9	Domain type contract
src/module/main/boycott-decision/utis/confidence.util.ts	959	24	Focused domain algorithm utility
src/module/main/boycott-decision/utis/entity-match.util.ts	6,345	189	Focused domain algorithm utility
src/module/main/boycott-decision/utis/reason.util.ts	1,733	36	Focused domain algorithm utility
src/module/main/boycott-provider/boycott-provider.controller.ts	732	30	HTTP controller / Elysia plugin
src/module/main/boycott-provider/boycott-provider.dto.ts	3,310	94	Runtime and TypeScript API contract
src/module/main/boycott-provider/boycott-provider.service.ts	13,472	437	Business logic service
src/module/main/boycott-provider/boycott-provider.test.ts	6,164	158	Bun unit/service test
src/module/main/brand/brand.controller.ts	896	50	HTTP controller / Elysia plugin
src/module/main/brand/brand.dto.ts	2,144	72	Runtime and TypeScript API contract
src/module/main/brand/brand.service.ts	4,348	135	Business logic service
src/module/main/brand/brand.test.ts	4,850	171	Bun unit/service test
src/module/main/organization/organization.controller.ts	1,619	60	HTTP controller / Elysia plugin
src/module/main/organization/organization.dto.ts	1,837	59	Runtime and TypeScript API contract
src/module/main/organization/organization.service.ts	4,360	130	Business logic service
src/module/main/organization/organization.test.ts	5,124	172	Bun unit/service test
src/module/main/product-provider/product-provider.dto.ts	917	25	Runtime and TypeScript API contract
src/module/main/product-provider/product-provider.service.ts	4,479	118	Business logic service
src/module/main/product-provider/product-provider.test.ts	2,465	79	Bun unit/service test
src/module/main/product/product.controller.ts	1,451	64	HTTP controller / Elysia plugin
src/module/main/product/product.dto.ts	4,238	111	Runtime and TypeScript API contract
src/module/main/product/product.service.ts	4,187	107	Business logic service
src/module/main/product/product.test.ts	4,745	181	Bun unit/service test
src/module/main/scan/scan.controller.ts	940	38	HTTP controller / Elysia plugin
src/module/main/scan/scan.dto.ts	1,632	48	Runtime and TypeScript API contract
src/module/main/scan/scan.service.ts	10,822	365	Business logic service
src/module/main/scan/scan.test.ts	12,314	384	Bun unit/service test
src/module/main/tenant/tenant.dto.ts	1,942	63	Runtime and TypeScript API contract
src/module/main/tenant/tenant.service.ts	4,548	130	Business logic service
src/module/main/tenant/tenant.test.ts	4,600	134	Bun unit/service test
src/module/main/user/user.controller.ts	636	25	HTTP controller / Elysia plugin
src/module/main/user/user.dto.ts	2,084	70	Runtime and TypeScript API contract
src/module/main/user/user.service.ts	3,192	97	Business logic service
src/module/main/user/user.test.ts	3,177	109	Bun unit/service test
src/module/organization/contact/contact.controller.ts	1,539	64	HTTP controller / Elysia plugin
src/module/organization/contact/contact.dto.ts	4,049	109	Runtime and TypeScript API contract
src/module/organization/contact/contact.service.ts	4,435	110	Business logic service

Path	Bytes	Lines	Role
src/module/organization/contact/contact.test.ts	3,708	133	Bun unit/service test
src/module/tenant/access/access.controller.ts	1,507	64	HTTP controller / Elysia plugin
src/module/tenant/access/access.dto.ts	2,086	75	Runtime and TypeScript API contract
src/module/tenant/access/access.service.ts	5,602	140	Business logic service
src/module/tenant/access/access.test.ts	6,099	210	Bun unit/service test
src/module/tenant/file/file.controller.ts	1,466	63	HTTP controller / Elysia plugin
src/module/tenant/file/file.dto.ts	1,730	62	Runtime and TypeScript API contract
src/module/tenant/file/file.service.ts	5,625	138	Business logic service
src/module/tenant/file/file.test.ts	3,865	142	Bun unit/service test
src/module/user/scan-history/scan-history.controller.ts	720	25	HTTP controller / Elysia plugin
src/module/user/scan-history/scan-history.dto.ts	1,260	35	Runtime and TypeScript API contract
src/module/user/scan-history/scan-history.service.ts	3,466	111	Business logic service
src/module/user/scan-history/scan-history.test.ts	2,670	92	Bun unit/service test
src/module/user/user-list/user-list.controller.ts	1,563	64	HTTP controller / Elysia plugin
src/module/user/user-list/user-list.dto.ts	1,855	62	Runtime and TypeScript API contract
src/module/user/user-list/user-list.service.ts	5,399	180	Business logic service
src/module/user/user-list/user-list.test.ts	4,584	162	Bun unit/service test
src/storage/client.storage.ts	351	10	Cloudflare R2 / S3 client
tsconfig.json	848	31	Strict TypeScript compiler policy

13. Source notes and official references

Repository claims are based on the local files. Technology comparisons were cross-checked against official vendor documentation accessed on 28 August 2026. URLs are printed in full for durability.

ID	Source	Used for	URL
S1	Bun documentation	All-in-one runtime, package manager, test runner, bundler	https://bun.com/docs
S2	Bun test runner	Jest-compatible built-in tests	https://bun.com/docs/test
S3	Bun Redis	RedisClient connection and reconnection behavior	https://bun.com/docs/runtime/redis
S4	Elysia lifecycle	Request phases and error hooks	https://elysiajs.com/essential/life-cycle
S5	Elysia validation	TypeBox-based runtime/type/OpenAPI schema	https://elysiajs.com/essential/validation
S6	Elysia plugins	Plugin composition and lifecycle scope	https://elysiajs.com/essential/plugin
S7	Elysia Eden	End-to-end type-safe client	https://elysiajs.com/eden/overview
S8	Drizzle data querying	SQL-like typed query philosophy	https://orm.drizzle.team/docs/data-querying
S9	Drizzle migrations	Codebase-first generated SQL workflow	https://orm.drizzle.team/docs/migrations
S10	Turso TypeScript reference	libSQL status, ORM fit, concurrency and transactions	https://docs.turso.tech/sdk/ts/reference
S11	Turso SDK introduction	Current package selection guidance	https://docs.turso.tech/sdk/introduction
S12	Turso Multi-DB Schemas	Deprecated managed schema feature	https://docs.turso.tech/features/multi-db-schemas
S13	Cloudflare R2 architecture	S3 compatibility, consistency and egress model	https://developers.cloudflare.com/r2/how-r2-works/
S14	Cloudflare R2 presigned URLs	Bearer-token semantics and signed operations	https://developers.cloudflare.com/r2/api/s3/presigned-urls/
S15	Docker multi-stage builds	Minimal artifact-only release stages	https://docs.docker.com/build/building/multi-stage/
S16	TypeScript strict	Stronger compiler correctness guarantees	https://www.typescriptlang.org/tsconfig/strict
S17	Zod	TypeScript-first runtime schemas	https://zod.dev/
S18	AI SDK Core	Provider abstraction, tools, structured output	https://ai-sdk.dev/docs/ai-sdk-core/overview
S19	VoltAgent overview	Agents, tools, memory, workflows, observability	https://voltage.dev/docs/overview/
S20	VoltAgent agents	Agent model and memory/tool capabilities	https://voltage.dev/docs/agents/overview/

13.1 Glossary

Term	Meaning in this project
Control plane	Global state and orchestration that manages users, organizations, tenant locations, and shared catalog.
Data plane	The tenant-specific databases where private operational data is stored.
DTO	Data transfer object schema defining request/response shape and validation.
Eden	Elysia client/type mechanism that carries server contract information to TypeScript clients.
libSQL	An SQLite fork/protocol and client ecosystem used by the project's Turso connections.
LRU	Least recently used eviction policy for bounded tenant connection caching.
Partial index	An index applying only to rows matching a WHERE predicate, such as active rows.
Presigned URL	Time-limited bearer URL authorizing one object-storage operation.
Saga	A multi-step distributed transaction modeled with durable progress and compensating actions.
Schema migration	A controlled transition from one database structure/version to another.
Soft delete	Marking a row deleted with a timestamp instead of physically removing it.
Stateless module	A request component whose durable truth is external, allowing replicas to serve interchangeably.
Tenant	A user or organization with an isolated database and access policy.
TypeBox	JSON-schema-oriented runtime schema system behind Elysia's t helper.
Zod	TypeScript-first runtime validation library used by the AI/tool layer.

End of guide. The repository remains the authoritative source for implementation details and future changes.